

Now I have comprehensive research data. Let me compose the final answer.

「文字→PPTX」LLM 簡報自動生成：問題分析、相關論文與開源專案全景

這是一個典型的**多模態結構化內容生成**問題，橫跨自然語言處理、版面設計、圖像檢索/生成等多個研究領域。以下針對你提出的五大核心問題，逐一梳理最相關的學術論文與開源專案，並進行架構層面的分析。

一、問題總覽與研究版圖

自動簡報生成（Automatic Presentation Generation）的 pipeline 可拆解為：

- 1. 內容擴充（Content Expansion / Summarization）
- 2. 內容→分頁分配（Content-to-Slide Allocation）
- 3. 文字排版定位（Text Layout Positioning）
- 4. 圖片搜尋/生成（Image Retrieval / Generation）
- 5. 圖文聯合排版（Joint Text-Image Layout）

現有研究大致分為三個典範：**規則式 (rule-based)**、**模板式 (template-based)**、**編輯式 (edit-based)** 以及最新的 **Agent 式 (agentic)**。以下按照你的五個問題進行深度分析。

二、核心論文與對應問題映射

論文 / 專案	發表場所	主要解決的問題	核心方法
PPTAgent (Zheng et al.)	EMNLP 2025	②③④⑤ 全流程	兩階段 edit-based：分析參考簡報 → 編輯式生成
PreGenie (Xu et al.)	EMNLP 2025 Findings	①②③⑤ 全流程	Slidev + 多 MLLM 協作 + Code Review + Page Review
DocPres (Bandyopadhyay et al.)	INLG 2024	①② 內容擴充與分配	多階段分層摘要 + Slide-to-Section 映射 + CLIP 圖配
DOC2PPT (Fu et al.)	AAAI 2022	①②③⑤	階層式 seq2seq + 改寫模組 + Layout Prediction MLP
D2S (Sun et al.)	NAACL 2021	①② 內容選取	Query-based 摘要 + 圖片自適應選取
SlideCoder (Tang et al.)	EMNLP 2025	③⑤ Layout 還原	Layout-aware RAG + 色彩梯度分割 + 階層式代碼生成

論文 / 專案	發表場所	主要解決的問題	核心方法
SlideSpawn (Kumar et al.)	arXiv 2024	①②④	MLP 句子重要度 + ILP 選取 + 聚類標題分組
LayoutNUWA (Tang et al.)	ICLR 2024	③⑤ Layout 生成	Layout → HTML Code Generation → LLM 填充
PosterLlama (Seol et al.)	ECCV 2024	⑤ Content-aware Layout	VLM + HTML 排版 + 深度增強策略
SEGA (Wang et al.)	ICCV 2025	⑤ Content-aware Layout	粗粒度→細粒度兩階段演進 + 設計先驗知識
LayoutTransformer (Gupta et al.)	ICCV 2021	③⑤	Self-attention 學習 layout 元素間的上下文關係
LayoutGAN (Li et al.)	ICLR 2019	③⑤	GAN + wireframe discriminator 直接生成結構化排版
PPTC Benchmark (Guo et al.)	arXiv 2023	全流程評估	279 多輪指令 Benchmark，測試 LLM 操控 PPTX 能力
Slide Retrieval Study (Giouroukis et al.)	arXiv 2025	④ RAG 投影片檢索	ColPali / Caption-based / Hybrid 混合檢索比較

三、針對五大問題的深度分析

問題一：內容擴充

這是從簡短輸入文字擴展成完整簡報內容的問題，本質上是**受控文本生成 (Controlled Text Generation)** 和**多層摘要的逆過程**。

DocPres 提出了最系統化的解法：先建立文件的「鳥瞰圖」(Bird's-eye View)——從子章節→章節→全文逐層摘要，然後用 Chain-of-Thought 從鳥瞰圖反向展開為 K 個投影片主題。這個階層式展開策略能有效避免 LLM 面對長文本時的 context 退化問題。 [themoonlight](#)

PPTAgent 走不同路線：它不從零擴充，而是以「文件分段 + 參考簡報 schema 匹配」的方式，將文件內容對應到已有的投影片結構中。Stage I 會提取參考簡報的 schema（每頁的 Category/Description/Data 結構），Stage II 的 Outline Generation 再將文件段落映射到對應的參考投影片。 [arxiv](#)

PreGenie 則用 Text Summarizer (LLM) 提取關鍵資訊（摘要、標題、作者等），搭配 Image Captioner (VLM) 為圖片標記標題與描述，兩者共同組成完整的內容素材。 [arxiv](#)

建議方案：採用 DocPres 的分層摘要策略作為內容擴充的骨幹，先建立全文的結構化理解，再讓 LLM 以 CoT 方式逐步展開各主題要點。

問題二：擴充後文字如何分配到各分頁

這是文件-投影片結構映射 (**Document-to-Slide Structural Mapping**) 的問題。

DocPres 的 Slide-to-Section Mapping 是最直覺的方案：用 LLM 為每個投影片標題匹配原始文件的章節，使用 Levenshtein 編輯距離（閾值 90%）來穩健匹配，有效防止 LLM 幻覺。

[themoonlight](#)

DOC2PPT 用更底層的方式解決：它設計了一個三層階層式 RNN (PTsec → PTslide → PTobj)，模型自動學習文件章節 → 投影片 → 頁面元素的對應關係，並透過 attention 機制決定哪些句子歸到哪頁。[arxiv](#)

PPTAgent 的 Outline Generation 階段則由 LLM 生成結構化大綱，每個大綱條目明確指定「參考哪張投影片」和「使用文件的哪些段落」，形成一對一的映射關係。[arxiv](#)

SlideSpawn 使用 ILP（整數線性規劃）來選取句子，以最大化重要性分數為目標，同時約束字數上限和句子間冗餘度，再用語意聚類將選中句子分組為不同投影片。[themoonlight](#)

建議方案：先用 LLM 生成投影片大綱（標題列表），再用語意相似度將文件段落映射到各標題。PPTAgent 的 schema-guided outline 或 DocPres 的 section mapping 都是可行路線。

問題三：每頁文字排版定位

這是版面生成 (**Layout Generation**) 的核心子問題。

LayoutNUWA 首創將 layout 生成視為**代碼生成任務**：把版面元素量化為 HTML/SVG 的 <rect> 標籤，用 LLM 透過 Code Instruct Tuning 預測各元素的 x, y, width, height。在多個 dataset 上達到 50%+ 的性能提升。[microsoft](#)

DOC2PPT 的 Layout Prediction 模組用兩層 MLP 預測每個投影片物件的 {lx, ly, lw, lh}（位置和尺寸），輸入是物件層級的隱藏表示和章節級別的 attention 向量。[tsujiifu.github](#)

PPTAgent 迴避了從零生成 layout 的困難——它從參考投影片**繼承**排版，透過 edit API (replace_span, del_span, clone_paragraph 等) 修改內容而保留原始佈局。HTML 渲染層讓 LLM 能精確操控元素位置。[arxiv](#)

SlideCoder 提出了 Color Gradient-based Segmentation 演算法，先從參考圖像分割出版面區塊，再用 Hierarchical RAG 為各區塊生成對應的代碼。它還發布了 SlideMaster（7B 模型）用於投影片代碼生成。[aclanthology](#)

LayoutTransformer 用 self-attention 學習 layout 元素之間的上下文關係，支援自回歸生成（從空集合或 partial layout 完成），且能自動捕捉元素的語義屬性。[arxiv](#)

建議方案：如果使用模板/參考投影片，PPTAgent 的 edit-based 方式最穩定。如需從零生成，LayoutNUWA 的 HTML 代碼生成範式與你的 python-pptx 工作流最相容，可將 HTML 座標轉換為 PPTX 座標。

問題四：搜尋或生成適合每頁的圖片

這涉及圖像檢索 (**Image Retrieval**) 和圖像生成 (**Image Generation**) 兩個方向。

DocPres 的圖像選取策略最完整：先提取文件中所有圖片，過濾不合適的（比例、重複 logo、尺寸），再用 CLIP embedding 計算圖片與投影片文字的 cosine similarity，取最高分且超過閾值（80%）的圖片。 [themoonlight](#)

DOC2PPT 引入了 text-figure matching objective，鼓勵相關的文字-圖片配對出現在同一頁投影片上，透過訓練讓模型學會圖文配對。 [arxiv](#)

PreGenie 在純文字輸入的情境下，用 LLM 生成圖像提示詞，呼叫外部 text-to-image 模型產生插圖；在文件輸入場景下，Image Captioner（VLM）為每張圖生成標題、描述和原文位置，供後續匹配。 [arxiv](#)

PPTAgent V2 在最新版本加入了 AI 圖像生成、Web 搜尋和 Agent Sandbox（30+ 工具），可自主建立視覺素材。 [github](#)

投影片檢索研究 (Giouroukis et al., 2025) 系統比較了多種 RAG 方式：ColPali（視覺 late-interaction）達到 86.9% NDCG@10 但存儲需求大；VLM captioning + hybrid retrieval 在精度與效率間取得最佳平衡（83.9% NDCG@10，存儲需求少 67%）。 [themoonlight](#)

建議方案：採用 CLIP 相似度做圖片匹配 + text-to-image model 做圖片生成的雙軌策略。RAG 部分可參考 captioning-based 檢索方式。

問題五：圖文聯合排版——最大難點

這是 **Content-Aware Layout Generation** 問題，也是學界目前最活躍的研究方向。

PosterLlama (ECCV 2024) 將版面元素格式化為 HTML code，利用 VLM 理解背景圖的語意後生成排版，搭配深度增強策略保證視覺美觀。支持無條件生成、條件生成、Layout 補全等多種模式。 [ai-scholar](#)

SEGA (ICCV 2025) 提出 Stepwise Evolution 範式：先用粗粒度模組估計初步排版，再用精調模組迭代細化。關鍵創新是將**設計先驗知識（design prior）** 注入模型，顯著提升複雜版面的成功率。 [iccv.thecvf](#)

PreGenie 的 Page Review 機制最直接解決圖文配合問題：Code Generator 先生成初版排版，Page Reviewer（VLM）逐頁檢查渲染結果——發現圖片溢出就調整尺寸、發現文字堆疊就改為分欄佈局、發現留白不均就重新組織為 bullet point 格式。平均需要 3.8 輪視覺審查迭代。 [arxiv](#)

Illustration Layout for Slides (ACM MM 2025) 專門處理投影片中插圖的佈局問題，用 pixel-based diffusion model 預測插圖應該放置的位置和大小，是目前唯一直接針對投影片插圖排版的研究。 [dl.acm](#)

建議方案：圖文聯合排版建議採用**兩階段策略**——先用 LLM/LayoutNUWA 生成初步版面，再用 VLM（如 Qwen2.5-VL）對渲染結果做視覺審查並迭代修正。PreGenie 的 Code Review + Page Review 雙迴圈是目前效果最好的工程方案。

四、推薦的開源專案

專案	Stars	特色	適用場景
PPTAgent	~2,000+	V2 支援 Deep Research + AI 圖片生成 + 30+ 工具沙箱	最完整的端到端方案
SlideCoder	—	Layout-aware RAG + SlideMaster 7B 模型	Layout 還原與代碼生成
txt2pptx	—	LangChain + RAG on python-pptx docs	輕量快速原型
Powerpointer-For-Local-LLMs	—	本地 LLM + python-pptx，7 種設計模板	本地部署、隱私優先
python-pptx	3,200+	PPTX 讀寫基礎庫	底層 PPTX 操控
Layout-Generation	—	LayoutVAE + LayoutTransformer + LayoutGAN 基線實現	版面生成研究
LayoutNUWA	—	Layout 作為 Code Generation 任務	學術研究 / Layout 模組嵌入

PPTAgent V2 是目前功能最完整的開源方案，其 Agent 架構包含 Planner、Content Organizer、Editor、Coder、Layout Selector 等多個角色，值得作為你的系統架構參考。
[blog.csdn](https://blog.csdn.net/qq_42699942/article/details/138000000)

五、針對你的 RAG 應用的架構建議

綜合所有論文的方法論，建議你的系統採用以下 pipeline：



關鍵工程建議：- 中間表示格式：PPTAgent 用 HTML 渲染 PPTX 的 XML，顯著降低 LLM 理解難

度（PPT XML 動輒上千行，HTML 表示可壓縮至數十行）[arxiv](#) - **Self-correction** 機制：
PPTAgent 和 PreGenie 都證明了 REPL 式的迭代修正對提升成功率至關重要（PPTAgent 從 74.6% 提升到 95%）[arxiv](#) - 視覺驗證迴圈：PreGenie 的實驗證明 Page Review 在設計美觀和頁面一致性上帶來最顯著提升，是區分「能用」與「好用」的關鍵[arxiv](#) - 評估框架：建議採用 PPTEval 的三維度評估（Content, Design, Coherence），搭配 CLIP/LongCLIP 評估圖文相關性[arxiv](#)

有，可以，而且有幾個真的很適合「投影片插圖」這種中小尺寸、偏實用場景的部署。

下面列的是「開源 + 相對中小型 / 高效率」的 text-to-image 模型與方向，你可以直接掛在你的 PPT 生成 pipeline 後面當圖片服務。

1. SSD-1B：縮小版 SDXL（推薦）

- 專案：Segmind 的 **SSD-1B** [github](#)
 - 類型：Stable Diffusion XL 的蒸餾版 latent diffusion 模型
 - 規模：約 **1.3B** 參數，比 SDXL 小 50%，推理速度快約 60% [wandb](#)
 - 授權：Apache 2.0（商用友善）[github](#)
 - 優點：
 - 針對速度和 VRAM 做過優化，適合 8–12GB GPU 或雲端小機器。[github](#)
 - 原生支援 diffusers，可以很容易整合進 Python / FastAPI 服務。[github](#)
 - 輸出品質對於「投影片用插圖」已足夠（教育、商業示意圖等）。[github](#)
 - 適合場景：你現在這種「每頁一張示意圖」「不追求 CG 級質感」的 PPT 生成。
-

2. SDXL Turbo / Lightning：極少步數快速生成（若你能接受稍大模型）

嚴格來說 SDXL Turbo 本身不是「小模型」，但步數極少，實際吞吐量對服務端很友善。

- **SDXL Turbo / SDXL Lightning**：
 - 特色：1–4 步就能出圖，主打 **real-time / 低延遲**。[reddit](#)
 - 適合：你用單張 512–768 px 圖，要求「快」大於「極致細節」時。
 - 實務上很多人用 Lightning 在 6–8 步就覺得品質夠用。[reddit](#)
 - 搭配策略：
 - 若 GPU 記憶體還可以，Turbo/Lightning 可以當「預設插圖引擎」；
 - 若你真的要壓 VRAM，就選 SSD-1B 這種縮小版。
-

3. 研究向的「輕量 Diffusion」方向（可作為壓縮參考）

如果你未來想自己壓縮模型或做定製版：

- **Toward Lightweight and Fast Decoders for Diffusion Models** [arxiv](#)
- 思路：只替換 Stable Diffusion 的 VAE 解碼器為輕量 ViT/Taming Transformer，UNet 保持不變，整體推理時間加速約 15%，解碼子模組可加速至 20 倍。 [themoonlight](#)
- 用途：在不改 UNet 的前提下，用輕量解碼器換取吞吐量，非常適合大規模產圖服務。 [themoonlight](#)
- **BK-SDM**：針對 Stable Diffusion 的架構壓縮與知識蒸餾，取得更小、更快的變體。 [arxiv](#)

這些論文不是「直接可用模型」，但可以作為你以後自行蒸餾 / 壓縮 text-to-image 的參考。

4. 如何在你的 PPT RAG 系統中實際使用

在你的系統中，圖片需求通常是「小型示意圖 / icon 風格」，可以採用這種策略：

1. 選模型：
2. 若你目標是「雲端或工作站」：SSD-1B 或 SDXL Lightning；
3. 若你有更嚴格的 VRAM 限制，再考慮專門的 512px 小模型（可從 HF 集合中挑，如 multimodalart 的 text-to-image collection）。 [huggingface](#)
4. Prompt 設計：
5. 自動從每頁 slide 的標題 + bullet points 抽關鍵詞；
6. 加上風格約束：「flat illustration, minimalistic, for presentation slide, white background」等；
7. 輸出控制：
8. 固定解析度（例如 768×512）以方便版面排版；
9. 在 metadata 中記錄 prompt / random seed，方便重產或後續微調。
10. 服務化：
11. 用 diffusers + SSD-1B 部署成獨立 microservice，LLM 只負責生成 prompt，不做圖片本身。

如果你願意，我可以下一步幫你設計一個「從 slide 結構 → 圖像 prompt → SSD-1B 推理 → 回填到 python-pptx」的具體介面與伺服器結構。